Image Compression Using Clustering
Qi(Sophie) Pan

1 Image compression using clustering

In this programming practice, you are going to apply clustering algorithms for image compression. This can also be viewed as an example of segmenting colors in an automated fashion using K-means clustering.

Your task is to implement *K-means* for this purpose. **It is required you implement the algorithms yourself rather than calling k-means from a package. However, it is ok to use standard packages for supplementary tasks, e.g., file i/o, linear algebra, and visualization.**

Formatting instruction

As a starting point, we suggest the following input/output signature for your k-means algorithm.

Input

- `pixels`: the input image representation. Each row contains one data point (pixel). For image dataset, it contains 3 columns, each column corresponding to Red, Green, and Blue components. Each component has an integer value between 0 and 255.
- `k`: the number of desired clusters.

Output

- `class`: cluster assignment of each data point in pixels. The assignment should be 1, 2, 3, etc. For $k = 5$, for example, each cell of the class should be either 1, 2, 3, 4, or 5. The output should be a column vector with `size(pixels, 1)` elements.
- `centroid`: location of k centroids (or representatives) in your result. With images, each centroid corresponds to the representative color of each cluster. The output should be a matrix with K rows and 3 columns. The range of values should be [0, 255], possibly floating point numbers.

Hand-in

Both of your code and report will be evaluated. Upload the code as a zip file, and the report as a pdf, separately from the zip file. In your report, answer the following questions:

Use k-means with squared-ℓ_2 norm as a metric for `hestain.bmp` and `football.bmp` and also choose a third picture of your own to work on. We recommend the size of 320×240 or smaller. Run your k-means implementation with these pictures, with several different $k = 2, 4, 8$.

Comment: `hestain.png` is an image of tissue stained with hematoxylin and eosin (H&E). This staining method helps pathologists distinguish between tissue types that are stained blue-purple and pink. Then the algorithm will segment the image into k regions in the RGB color space. For each pixel in the input image, the algorithm returns a label corresponding to a cluster.

Run your k-means implementation (with squared-ℓ_2 norm) with random initialization centroids. Please try multiple times and report the best one (in terms of image quality).

Please write in your report how long it takes to converge for each k (report the number of iterations and elapsed time in seconds) and also include the resulting compressed pictures for each k **in the report**.

Note

- You may see errors caused by empty clusters when you use too large k. Your implementation should treat this exception as well. That is, do not terminate even if you have an empty cluster, but automatically decrement to a smaller number of clusters.
- We recommend you test your code with several different pictures so that you can detect some problems that might happen occasionally.
- If we detect plagiarism from any other student's code or from the web, you will not be eligible for any credit for the entire homework, not just for this problem.

Solution:

1. For reproducibility, the random seeds=2, 17, 45, 73 for each k=2, 4, 8 are chosen.

2. Different seed(random initialization centroids) gives different results. But this is only obvious when k is bigger. When k=2, the difference is super subtle. We can not even use our eyes to tell.

3. The bigger the k, the longer the running time, also with more iteration times. The WCSS (total sum of squared distance) also decreases. With a bigger k, the compressed images look much nicer and more colorful! This fits my expectation. For example, when k=2, we only have 2 colors to represent a picture, but when k=8, we have 8 colors to represent it!

Overall performance of all 3 pictures:

Average time, iterations and total sum of squared distance for each picture, each k				
Picture	k	avg number of iterations	avg time use (sec)	avg total sum of squared distance
football	2	21	2.05	1241175018
	4	30	3.24	480195656
	8	86	10.40	248957552
hestain	2	30	0.48	174601177
	4	42	0.84	73288650
	8	68	2.04	34728331
krumlov	2	18	0.75	733479383
	4	27	1.47	223595915
	8	93	7.40	91709376

Now, let's take a look at the details of each picture.

Picture compression for "football.bmp"

Football Picture: k=2 with random seed=2,17,45,73				
k	random seed	number of iterations	time use (sec)	total sum of squared distance
k=2	2	25	2.47	1241175017.828861
	17	16	1.56	1241175017.828861
	45	23	2.30	1241175017.828861
	73	19	1.86	1241175017.828861

We can't really tell the difference between pictures with different seeds when k=2.

Football Picture: k=4 with random seed=2,17,45,73				
k	random seed	number of iterations	time use (sec)	total sum of squared distance
k=4	2	27	3.09	510206785.6753904
	17	30	2.99	470191946.49104124
	45	36	3.94	470191946.49104124
	73	26	2.94	470191946.49104124

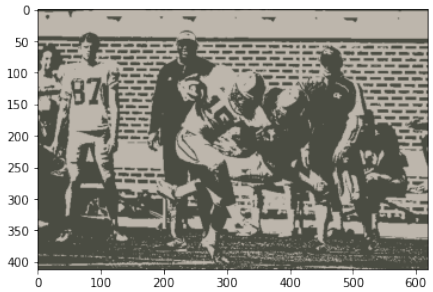


Figure 1: football with seed=2, k=2

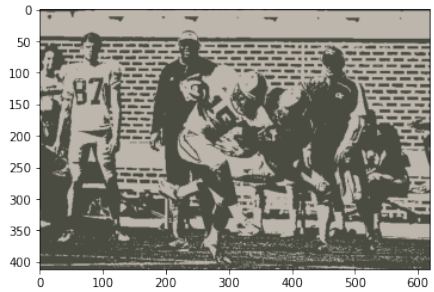


Figure 2: football with seed=17, k=2

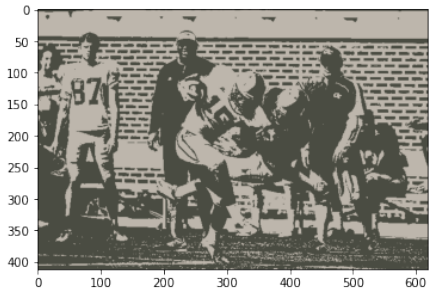


Figure 3: football with seed=45, k=2

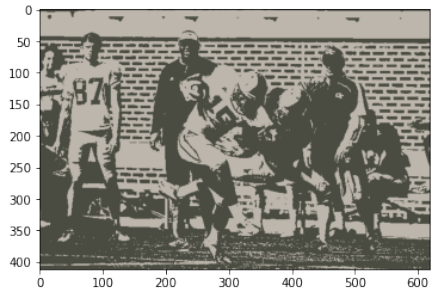


Figure 4: football with seed=73, k=2

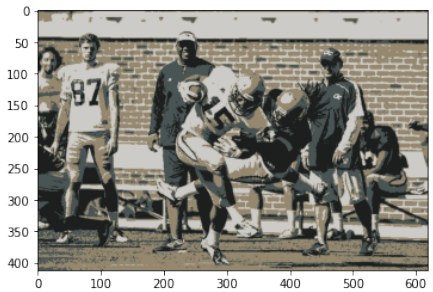


Figure 5: football with seed=2, k=4

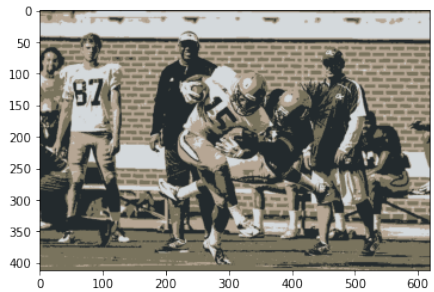


Figure 6: football with seed=17, k=4

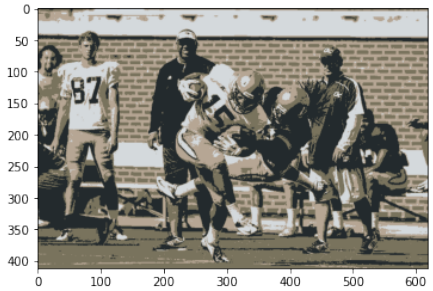


Figure 7: football with seed=45, k=4

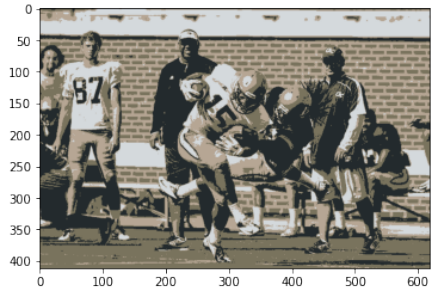


Figure 8: football with seed=73, k=4

The first picture with seed=2, k=4 looks a bit nicer than the other 3 pictures since the color of the player is blueish. But for the total sum of squared distance, the other 3 pictures are with less distance. They should be better regarding our objective function of Kmeans (min the sum of total squared of norm 2 distance for each point).

Football Picture: k=8 with random seed=2,17,45,73				
k	random seed	number of iterations	time use (sec)	total sum of squared distance
k=8	2	132	16.45	263550808.14715844
	17	44	5.24	234364168.9586848
	45	52	6.34	234364422.25132793
	73	114	13.58	263550808.14715844

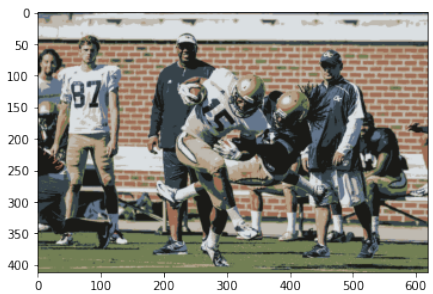


Figure 9: football with seed=2, k=8

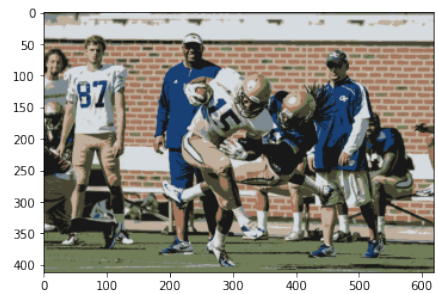


Figure 10: football with seed=17, k=8

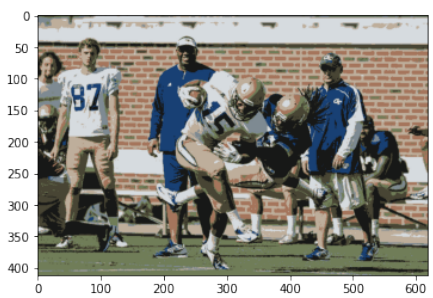


Figure 11: football with seed=45, k=8

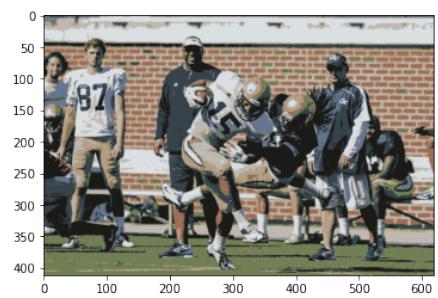


Figure 12: football with seed=73, k=8

With k=8, the pictures look much nicer than k=2 and 4, since they are more colorful. When comparing the seeds, for seed=45 and seed=17, the pictures look nicer with more blue on the players' T-shirts. The running time and iteration numbers are also much less than the seeds =2 and 73.

Picture compression for 'hestain.bmp'

Hestain Picture: k=2 with random seed=2,17,45,73				
k	random seed	number of iterations	time use (sec)	total sum of squared distance
k=2	2	31	0.51	174601176.53804392
	17	28	0.46	174601176.53804392
	45	28	0.45	174601176.53804392
	73	31	0.48	174601176.53804392

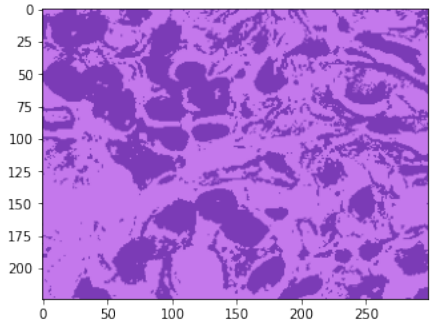


Figure 13: hestain with seed=2, k=2

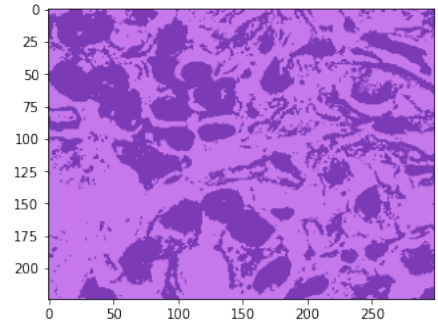


Figure 14: hestain with seed=17, k=2

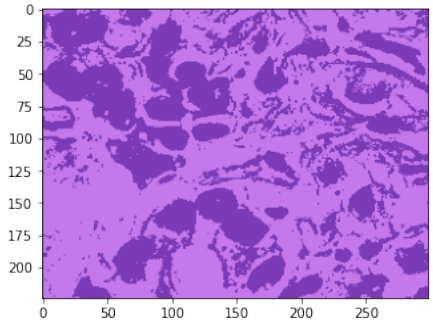


Figure 15: hestain with seed=45, k=2

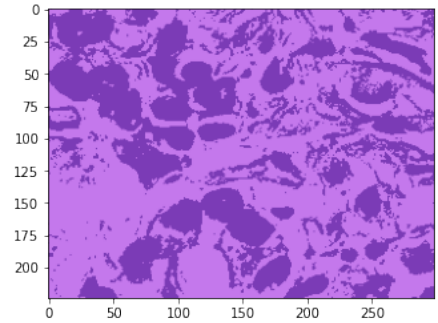


Figure 16: hestain with seed=73, k=2

There isn't difference between the pictures. The numbers of the total sum of squared distances are all the same. And I have double checked the centers, they are all the same.

Hestain Picture: k=4 with random seed=2,17,45,73				
k	random seed	number of iterations	time use (sec)	total sum of squared distance
k=4	2	48	0.98	73288649.82298338
	17	32	0.65	73288650.28439674
	45	56	1.11	73288650.28439674
	73	30	0.61	73288649.82298338

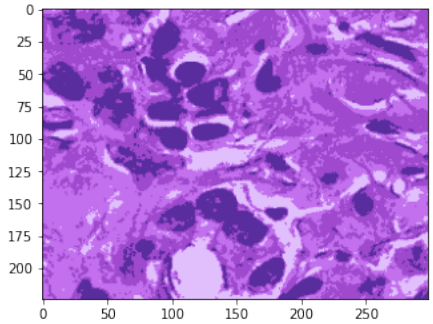


Figure 17: hestain with seed=2, k=4

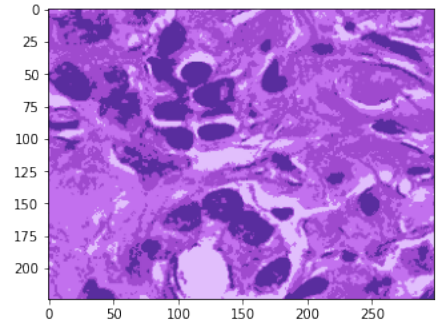


Figure 18: hestain with seed=17, k=4

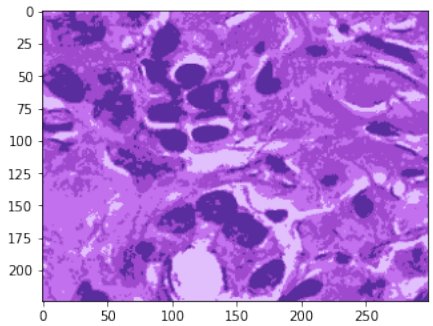


Figure 19: hestain with seed=45, k=4

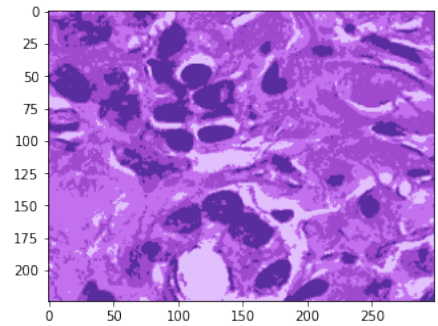


Figure 20: hestain with seed=73, k=4

When k=4, the pictures are with much more details compared with k=2. The edges of different tissue's boundary are much more clear. I prefer pictures with seed=2 and 73 with less WCSS.

Hestain Picture: k=8 with random seed=2,17,45,73				
k	random seed	number of iterations	time use (sec)	total sum of squared distance
k=8	2	91	2.60	34896066.70517948
	17	71	2.16	34920654.735939026
	45	41	1.28	34175949.90815069
	73	67	2.13	34920654.735939026

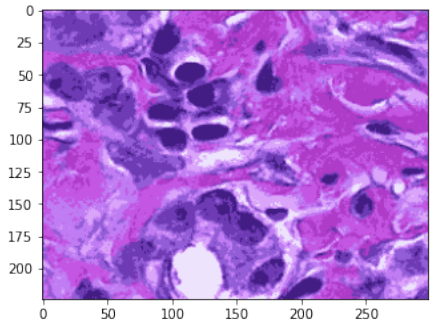


Figure 21: hestain with seed=2, k=8

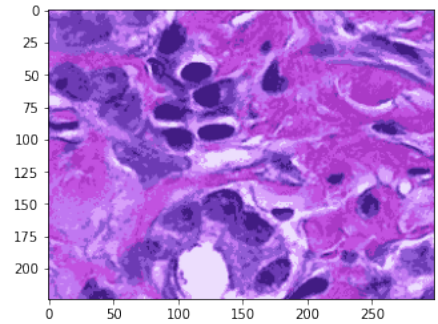


Figure 22: hestain with seed=17, k=8

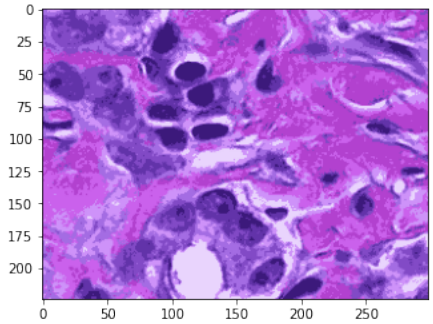


Figure 23: hestain with seed=45, k=8

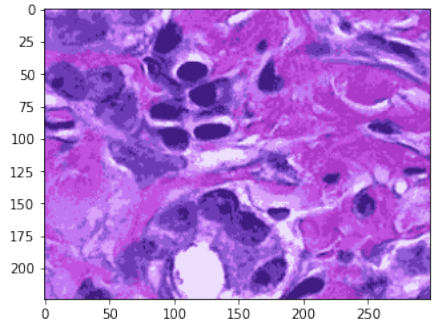


Figure 24: hestain with seed=73, k=8

When k=8, the pictures look more granular. And we can tell there begins to show some obvious difference. Like when comparing pictures of seed=2 vs seed=45, there is not a clear boundary for that area at the most left middle part of the picture with seed=45. However, in the picture with seed=2, the boundary is very clear. So when regarding WCSS, I should prefer picture with seed=45, but for picture quality, I prefer seed=2.

Picture compression for my own picture "krumlov.jpeg"
This is my own picture which I just took in middle Europe during my winter break in 2022. LOVE THAT PLACE! :-)

Krumlov Picture: k=2 with random seed=2,17,45,73				
k	random seed	number of iterations	time use (sec)	total sum of squared distance
k=2	2	16	0.65	733479383.3561454
	17	16	0.66	733479383.3561454
	45	18	0.76	733479324.2349342
	73	22	0.92	733479383.3561454

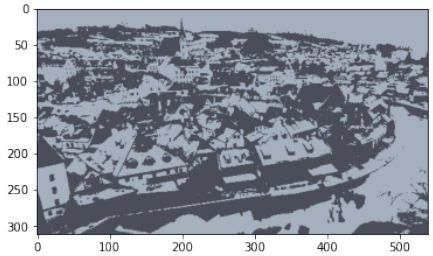


Figure 25: krumlov with seed=2, k=2

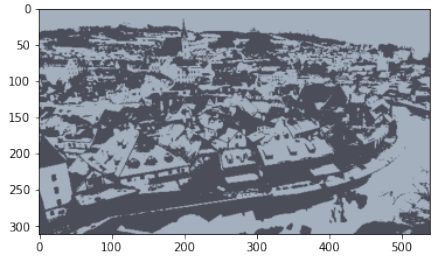


Figure 26: krumlov with seed=17, k=2

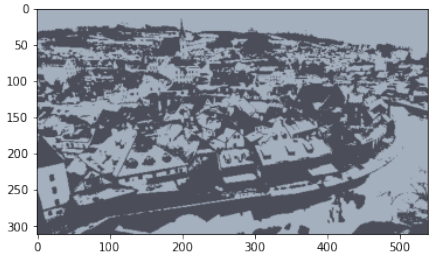


Figure 27: krumlov with seed=45, k=2

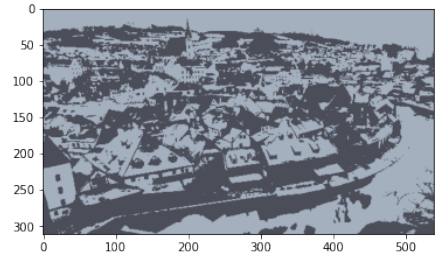


Figure 28: krumlov with seed=73, k=2

I can't tell the difference between them at all.

Krumlov Picture: k=4 with random seed=2,17,45,73				
k	random seed	number of iterations	time use (sec)	total sum of squared distance
k=4	2	25	1.43	223595914.7383222
	17	28	1.43	223595914.7383222
	45	31	1.62	223595914.7383222
	73	24	1.39	223595914.7383222

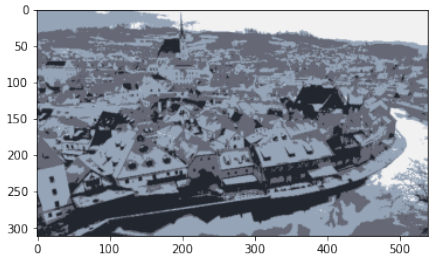


Figure 29: krumlov with seed=2, k=4

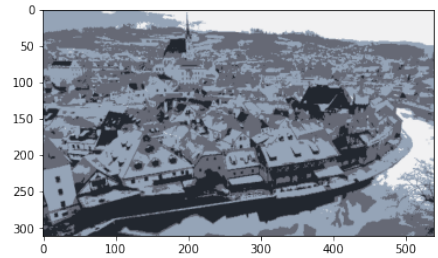


Figure 30: krumlov with seed=17, k=4

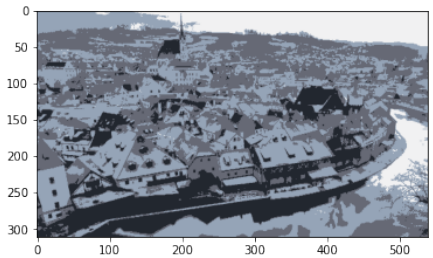


Figure 31: krumlov with seed=45, k=4

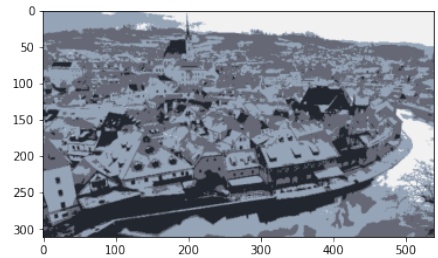


Figure 32: krumlov with seed=73, k=4

Very interesting that I got 4 pictures with 4 different seeds with the same WCSS. And I can't tell the difference between different seeds. But the pictures' quality improve.

Krumlov Picture: k=8 with random seed=2,17,45,73				
k	random seed	number of iterations	time use (sec)	total sum of squared distance
k=8	2	73	5.70	91708763.3428071
	17	146	11.51	91711672.02942546
	45	72	5.95	91708305.3898874
	73	80	6.44	91708763.3428071

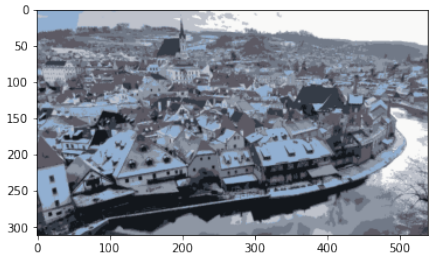


Figure 33: krumlov with seed=2, k=8

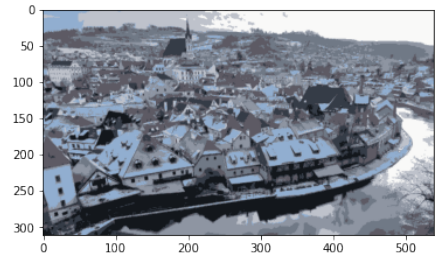


Figure 34: krumlov with seed=17, k=8

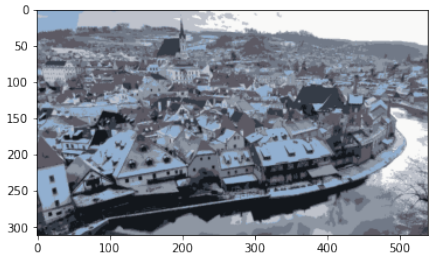


Figure 35: krumlov with seed=45, k=8

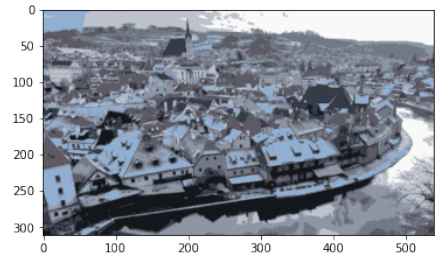


Figure 36: krumlov with seed=73, k=8

When k=8, we can tell the difference between these 4 pictures. For example, the upper left corner of pictures with seed=17 and seed=2. The more k we have, the pictures become more vivid and colorful.

2 MNIST Dataset clustering

This question is to compare different classifiers and their performance for multi-class classifications on the complete MNIST dataset at `http://yann.lecun.com/exdb/mnist/`. You can find the data file **mnist_10digits.mat** in the homework folder. The MNIST database of handwritten digits has a training set of 60,000 examples and a test set of 10,000 examples. Use the number of clusters $K = 10$.

We suggest you "standardize" the features (pixels in this case) by dividing the values of the features by 255 (thus mapping the range of the features from [0, 255] to [0, 1]).

We are going to use *purity* score as a performance metric: each cluster is assigned to the class which is most frequent in the cluster, and then the accuracy of this assignment is measured by the number of correlated assigned samples and divided by the size of the cluster:

$$\text{purity}_i = \frac{\text{corrected assigned samples}_i}{\text{size of cluster}_i}$$

for the cluster i.

1. Use the squared-ℓ_2 norm as a metric for clustering (you may base it on the code you had for Question 2 and make necessary changes.) Report the *purity* score for each cluster.

2. Now try your k-means with the Manhattan distance (or ℓ_1 distance) and repeat the same steps in Part (1). Please note that the assignment of data points should be based on the Manhattan distance, and the cluster centroid (by minimizing the sum of deviance – as a result of using the Manhattan distance) will be taken as the "median" of each cluster. Report the *purity* score for each cluster. Comment on which metric gives the better result?

Solution for Question 2.1 (l2 norm)
The shape of 'xtrain' is 60000*784 with each row as a picture. There are total of 60000 pictures with each picture 28*28. The real label 'ytrain' dataset also contains 60000 labels for each picture.

Note:
1. The overall running time for question 3 is long. In this section, I have tried many seeds and seems seed=12 is good. The worst one costs me 848 seconds with 164 iterations!!
2. Seed=12: The total iteration number is 43 and costs 217.7 seconds around.
3. For calculating the avg purity rate of the whole dataset, I summed up the count of all the majority digits within each cluster and divided it by the total sample count which is 60000.

$$avg\ purity\ rate = \frac{\sum_{i=0}^{9} major\ count_i}{60000}$$

The main takeaway:
1. The real label of number is not the same as the cluster number. As our TA posted on ED, it is a clustering problem, not classification. So in my result, some digits like 5 and 9 don't have any cluster with their major element as 5 or 9. But for number 0, there are two clusters whose major elements are 0.
2. Some digits' clusters are with very high purity scores like digit 0. One cluster of 0's purity score=90.73% and the other purity score=78.01%. Some other digits' clusters are with very low purity scores. For digit 4, its purity score=35.72%, and for digit 7, it is 42.68%. This might be because sometimes, people's handwriting of 4 is very similar to 9, the same as 1 vs 7.

Details of the 10 clusters with seed=12, l2 norm			
cluster number	real digit	size of the cluster	purity score
0	4	8919	0.3572
1	6	5717	0.8594
2	8	6516	0.5312
3	0	3206	0.7801
4	7	8840	0.4268
5	3	7429	0.5279
6	1	5651	0.5268
7	1	5959	0.6239
8	2	4668	0.8974
9	0	3095	0.9073
The avg purity score of the whole 'xtrain' dataset (l2 norm): 0.5908			

Solution for Question 2.2 (l1 norm)
Note:
1. The overall running time for l1 norm is even longer than l2 norm. I am still using seed=12 this time. The total iteration number is 85 and costs around 502.2 seconds.

The main takeaway:

1. The avg purity rate of the whole dataset for l1 norm is 0.5632 which is nearly the same as l2 norm. At this point, we can't tell which norm is definitely better.

2. Since the distance function is different, the sizes of each cluster are different compared with the l2 norm. But the purity scores within each cluster are similar.

Comparing the best and worst purity scores for the real digits in both l1 and l2 norm :

- l2 norm: the real digit cluster whose purity $> 70\%$: 0, 2, 6
- l1 norm: the real digit cluster whose purity $> 70\%$: 0, 2, 6
- l2 norm: the real digit cluster whose purity $< 50\%$: 4, 7
- l1 norm: the real digit cluster whose purity $< 50\%$: 4, 1

Seems the algorithm really struggles with numbers 1, 7, 4, 5, 9. For both l1 norm and l2 norm, there isn't any cluster for 5 and 9!

Details of the 10 clusters with seed=12, l1 norm			
cluster number	real digit	size of the cluster	purity score
0	4	8480	0.4441
1	6	4751	0.8196
2	8	4720	0.6288
3	0	2986	0.8212
4	7	8558	0.5102
5	3	7207	0.5532
6	1	10144	0.3426
7	1	6739	0.4802
8	2	3469	0.8735
9	0	2946	0.8876
The avg purity scorehw1 of the whole 'xtrain' dataset (l1 norm): 0.5632			

3 Political blogs dataset

We will study a political blog dataset first compiled for the paper Lada A. Adamic and Natalie Glance, "The political blogosphere and the 2004 US Election", in Proceedings of the WWW-2005 Workshop on the Weblogging Ecosystem (2005). It is assumed that blog-site with the same political orientation are more likely to link to each other, thus, forming a "community" or "cluster" in a graph. In this question, we will see whether or not this hypothesis is likely to be true based on the data.

- The dataset `nodes.txt` contains a graph with $n = 1490$ vertices ("nodes") corresponding to political blogs.
- The dataset `edges.txt` contains edges between the vertices. You may remove isolated nodes (nodes that are not connected to any other nodes) in the pre-processing.

We will treat the network as an undirected graph; thus, when constructing the adjacency matrix, make it symmetrical by, e.g., set the entry in the adjacency matrix to be one whether there is an edge between the two nodes (in either direction).

In addition, each vertex has a 0-1 label (in the 3rd column of the data file) corresponding to the true political orientation of that blog. We will consider this as the true label and check whether spectral clustering will cluster nodes with the same political orientation as possible.

1. Use spectral clustering to find the $k = 2, 5, 10, 25$ clusters in the network of political blogs (each node is a blog, and their edges are defined in the file `edges.txt`). Find majority labels in each cluster for different k values, respectively. For example, if there are $k = 2$ clusters, and their labels are $\{0, 1, 1, 1\}$ and $\{0, 0, 1\}$ then the majority label for the first cluster is 1 and for the second cluster is 0. **It is required you implement the algorithms yourself rather than calling from a package.**

   Now compare the majority label with the individual labels in each cluster, and report the *mismatch rate* for each cluster, when $k = 2, 5, 10, 25$. For instance, in the example above, the mismatch rate for the first cluster is 1/4 (only the first node differs from the majority), and the second cluster is 1/3.

   Solution for Question 3.1

   Note: I have tried coding with both the graph Laplacian matrix L and its variant matrix B. My report for Question 3.1 is based on graph Laplacian matrix L. For Question 3.2, I evaluated the best k based on both B and L matrices.

   Pre-processing for data: there are total of 1490 nodes and 266 nodes are without any edges. 3 of the useful nodes have connected to themselves. This means on the diagonal of matrix A, 3 elements are not 0. I have corrected all these issues with a symmetrical adjacent matrix A which is 1224*1224, with all values 0 or 1, and with all 0 values on the diagonal of A.

   Reference: spectral clustering.

   1. Because of the randomness, each time, the result is slightly different. I choose to use seed=73 for the following report for k=2,5,10,25.

   2. For k=2, 5, and 10, the overall mismatch rate is pretty bad which is around 46% to 48%. The majority points are concentrated within a few clusters. For example, for k=5, there are 1212 points squeezed in cluster 2. For clusters 0 and 3, there are only 2 points. For clusters 1 and 4, there are only 4 points.

   3. The low mismatch rate within clusters which are very small doesn't mean anything. Like if there are only two points within a cluster, a 0% mismatch rate doesn't really contribute to the overall mismatch rate for the whole graph.

   4. When k=25, the points are separated much more within these clusters and the overall mismatch rate greatly decreases to around 4.5%. This is great! Please also check the plot below with the details for question 4.2.

   5. Generally speaking, if the k is very big, like k is equal to the size of the whole dataset, each data point forms a cluster. As a result, the avg mismatch rate should be 0 at this extreme case. So my observation here really matches this expectation on theory: with bigger and bigger of k, the overall avg mismatch rate drops. But we need to keep in mind that the Kmeans algorithm is with randomness, so together with the increasing of k, some oscillation of the mismatch rate is normal.

Spectral clustering mismatch rate comparation for k=2,5,10,25 (Seed=73)					
k	cluster number	Majority label	cluster size	mismatch rate within cluster	overall mismatch rate
k=2	0	1	1222	0.47954	0.47876
	1	0	2	0.0	
k=5	0	0	2	0.0	0.47712
	1	1	4	0.0	
	2	1	1212	0.48020	
	3	0	2	0	
	4	0	4	0.5	
k=10	0	0	6	0.0	0.46405
	1	0	2	0.0	
	2	0	6	0.0	
	3	1	1135	0.48546	
	4	1	2	0.0	
	5	0	4	0.0	
	6	1	2	0.0	
	7	1	4	0.0	
	8	0	2	0.0	
	9	1	61	0.27869	
k=25	0	1	20	0.0	0.04493
	1	0	3	0.33334	
	2	1	2	0.0	
	3	0	15	0.2	
	4	0	1	0.0	
	5	1	590	0.04237	
	6	0	2	0.0	
	7	0	1	0.0	
	8	0	1	0.0	
	9	0	2	0.0	
	10	0	2	0.0	
	11	0	8	0.0	
	12	1	5	0.2	
	13	0	1	0.0	
	14	0	1	0.0	
	15	1	29	0.24138	
	16	0	2	0.0	
	17	0	4	0.5	
	18	0	482	0.02905	
	19	1	1	0.0	
	20	0	1	0.0	
	21	0	2	0.0	
	22	0	2	0.0	
	23	0	45	0.04444	
	24	0	2	0.0	

2. Tune your k and find the number of clusters to achieve a reasonably small *mismatch rate*. Please explain how you tune k and what is the achieved mismatch rate. Please explain intuitively what this result tells about the network community structure.

   Solution for Question 3.2

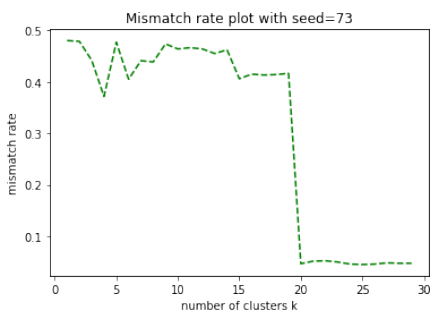
   Using graph laplacian matrix L:

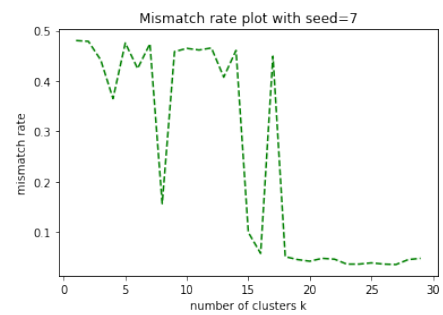
I have run my code from k=1 to 30 and the lowest mismatch rate happens around k=20. Since the randomness, I have tried to set the random seed with 2,7,73. Although the plots are a bit different, the whole trend is:

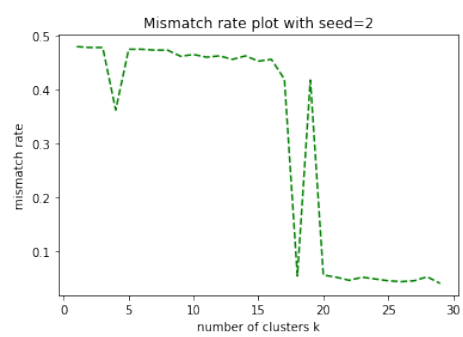
1. With k becoming larger, the total mismatch rate becomes lower and lower.

2. The best overall mismatch rate is around 0.045 within k=1,2,...,30

3. The overall mismatch rate drops greatly around k=20. With different seeds, there might be a few drops happening before k=20, but it is super unstable. So we should ignore these drops happening before k=20.







Using the variant matrix B, also called 'Normalized Graph Laplacian' :

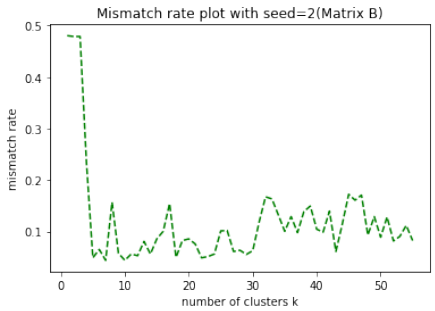
$$B = D^{-1/2} A D^{-1/2}$$

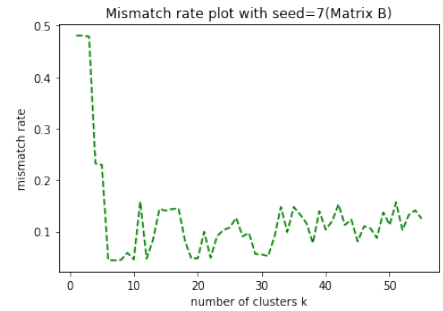
This approach ends with a little bit different when compared with using matrix L. Here are some interesting points I find:

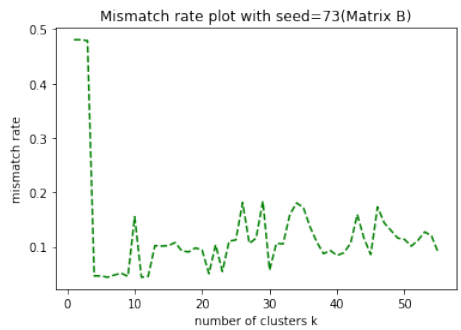
1. The best (lowest) mismatch rate happens around k=4 to k=6 which happens much faster than the 1st approach (around k=20).

2. The best overall mismatch rate is around 0.045 which is the same as the 1st approach.

3. After k=10, the mismatch rate is not very stable. It oscillates around 0.1. But it never jumps too high, not exceeding 0.2.







4 Math of k-means clustering

Given m data points x^i, $i = 1, \ldots, m$, K-means clustering algorithm groups them into k clusters by minimizing the distortion function over $\{r^{ij}, \mu^j\}$

$$J = \sum_{i=1}^{m} \sum_{j=1}^{k} r^{ij} \|\mathrm{x}^i - \mu^j\|^2, \tag{1}$$

where $r^{ij} = 1$ if x^i belongs to the j-th cluster and $r^{ij} = 0$ otherwise.

1. Derive mathematically that using the squared Euclidean distance $\|\mathrm{x}^i - \mu^j\|^2$ as the dissimilarity function, the centroid that minimizes the distortion function J for given assignments r^{ij} are given by

$$\mu^j = \frac{\sum_i r^{ij} \mathrm{x}^i}{\sum_i r^{ij}}.$$

That is, μ^j is the center of j-th cluster.

Solution for Question 4.1

$$J = \sum_{i=1}^{m} \sum_{j=1}^{k} r^{ij} (x^i - \mu^j)^T \cdot (x^i - \mu^j)$$

Take the partial derivative of J with each center μ^j, and at the optimal min value, the partial derivative should be 0. So we have:

$$\frac{\partial J}{\partial \mu^j} = \sum_{i=1}^{m} r^{ij} (-2x^i + 2\mu^j) = 0$$

$$\sum_{i=1}^{m} r^{ij} \mu^j = \sum_{i=1}^{m} r^{ij} x^i$$

$$\mu^j = \frac{\sum_{i=1}^{m} r^{ij} x^i}{\sum_{i=1}^{m} r^{ij}}$$

2. Derive mathematically what should be the assignment variables r^{ij} be to minimize the distortion function J, when the centroids μ^j are fixed.

   Solution for Question 4.2

   When μ^j are fixed, for each x^i, it will be assigned to only one label (belongs to only one cluster). So we have: $r^{ij} \in \{0, 1\}$ and $\sum_{j=1}^{k} r^{ij} = 1$. So for each point, we should choose the label that min the distance between the point and all the centers μ^j. So we have:

$$r^{ij} = 1,\ j = argmin \|\mathrm{x}^i - \mu^j\|^2,\ j \in \{1, 2, ..., k\}$$

$$r^{ij} = 0,\ for\, all\, other\, j$$